

REAL-TIME FACE RECOGNITION

Using Python And OpenCV

AQUIB JAVED KHAN

A real-time face-recognition system is capable of identifying or verifying a person from a video frame. To recognise the face in a frame, first you need to detect whether the face is present in the frame. If it is present, mark it as a region of interest (ROI), extract the ROI and process it for facial recognition.

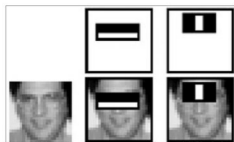


Fig. 1: Screenshot of Haar features

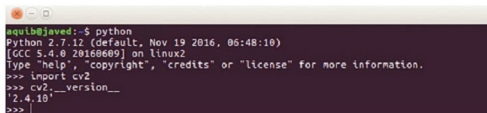


Fig. 2: Checking OpenCV using import command

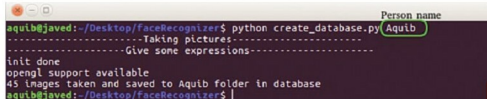


Fig. 3: Creating the database

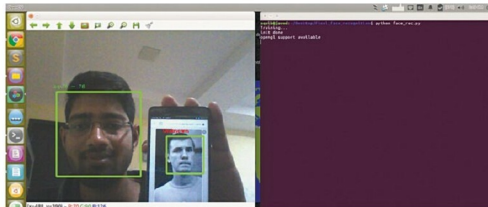


Fig. 4: Screenshot of face detection

Software

This project is divided into two parts: creating a database, and training and testing.

Creating a database. Take pictures of the person for face recognition after running create_database.py script. It automatically creates Train folder in Database folder containing the face to be recognised. You can change the name from Train to the person's name.

While creating the database, the face images must have different expressions, which is why a 0.38-second delay is given in the code for creating the data set. In this exam-

ple, we take about 45 pictures/images and extract the face, convert it into gray scale and save it to the database folder with its name.

Training and testing. Training and face recognition is done next. face_rec.py code does everything. The algorithm used here is Local Binary Patterns Histograms (LBPH).

Face detection is the process of finding or locating one or more human faces in a frame or image. Haar-like feature algorithm by Viola and Jones is used for face detection. In Haar features, all human faces share some common properties. These regularities may be matched using Haar features, as shown in Fig. 1.

Two properties common to human faces are:

1. The eye region is darker than the upper cheeks.
2. The nose bridge region is brighter than the eyes.

Composition of two properties forming matchable facial features are:

1. Location and size including eyes, mouth and bridge of nose
2. Value for oriented gradients of pixel intensities.

For example, the difference in brightness between white and black rectangles over a specific area is given by:

$$\text{Value} = \Sigma (\text{pixels in black area}) - \Sigma (\text{pixels in white area})$$

The above-mentioned four features matched by Haar algorithm are compared in the image of a face shown on the left of Fig. 1.

Testing procedure

Install OpenCV and Python on Ubuntu 16.04. The project was tested on Ubuntu 16.04 using OpenCV 2.4.10. When you run the following shell script, it will install all dependencies